

Desktop AI Assistant with Validated Screen-Based Task Guidance

Final Year Project Proposal

Faculty of Computing & Information Technology, Department of Software Engineering,
International Islamic University, Islamabad

Topic: Desktop AI Assistant with Validated Screen-Based Task Guidance (*working title, to be finalized*)

Group Members

- <1st Student Name>, <Registration Number>, <Email Address>
- <2nd Student Name>, <Registration Number>, <Email Address>

Supervisor: <Supervisor Name>

Semester: FALL-2026

Date of Submission: <DATE OF SUBMISSION>

1. Introduction

Modern users switch between many applications while working, and mistakes in code, documents or spreadsheets are often noticed only after significant time has been lost. General-purpose AI chatbots can help, but they require the user to stop working, copy content into a chat window and describe the problem manually.

This project proposes a **Desktop AI Assistant** that works alongside the user. The user selects a specific application window, and the assistant monitors it while the user works. It detects which supported application is in use, extracts the relevant task information, passes it through a **validation layer**, and only then sends the validated information to an AI model. The assistant then points out likely mistakes, explains them and offers suggestions through chat, desktop notifications and an interactive desktop avatar.

Beyond task guidance, the assistant offers conversational interaction, personalized memory, one-time and recurring reminders, and separate user accounts.

Project category: Student's own innovative idea, with a research component (evaluation of the reliability of AI suggestions when they are based on validated, application-specific information rather than raw screen content).

2. Objectives

1. Develop a desktop assistant that monitors only a user-selected application window and detects the application and task in progress.
2. Design and implement a validation mechanism that filters and verifies extracted information before it reaches the AI model, in order to reduce incorrect suggestions.
3. Provide accurate suggestions, mistake detection and explanations for a limited set of supported applications and tasks.
4. Provide an interactive desktop avatar, chat interface, personalized memory and reminders.
5. Support separate user accounts with isolated memory, reminders and settings.
6. Evaluate the system using measurable metrics (precision, recall, F1-score, false positive/negative rates and response time) on a predefined set of test scenarios.

3. Background / Existing System

3.1 Existing products

Product type	Examples	Limitation relevant to this project
Chat-based AI assistants	ChatGPT, Claude, Gemini	The user must manually copy content and describe the problem; no awareness of the current screen.
In-application assistants	GitHub Copilot, Microsoft 365 Copilot	Tied to a single vendor/application; no cross-application assistant, no avatar or personal memory model, and little transparency about reliability.
Voice/virtual assistants	Siri, Google Assistant, Cortana	Focus on commands and queries, not on observing and correcting ongoing desktop work.
Computer-use / GUI agents	Research agents and recent "computer use" models	Aim to perform actions autonomously from raw screenshots; accuracy is still limited and errors are hard to measure.

3.2 Problem statement

Existing assistants either wait for the user to ask or rely on raw screenshots interpreted directly by a large model. Raw-screen interpretation is prone to misreading content and to confident but incorrect suggestions (hallucinations). There is no widely available lightweight desktop assistant that (a) watches a user-chosen window, (b) validates what it sees against application-specific rules before consulting an AI model, and (c) stays silent when it is not confident.

3.3 Key concepts

- **Screen-based task understanding:** inferring what the user is doing from visual or structural information on screen.
- **Application/task detection:** identifying the active application and the type of task being performed.
- **Validation layer:** a rule-based or model-based check that confirms extracted information is relevant and reliable before it is sent to the AI model.
- **Precision, recall, F1-score:** standard measures of how many detected mistakes are correct (precision), how many real mistakes are found (recall), and their harmonic mean (F1).

3.4 Research gap

Most related work evaluates agents that act on screens end to end. Less attention has been given to measuring whether *application-specific validation* placed before the language model improves the reliability of assistive suggestions. This project addresses that gap (final research gap statement to be refined after the literature review).

4. Proposed Solution

4.1 Approach

The system is an Electron desktop application with a Node.js backend. The processing pipeline is:

1. **Window selection:** the user chooses the application window to monitor and can start or stop monitoring at any time.
2. **Screen capture:** the selected window is captured periodically (capture method to be finalized, for example Electron's desktopCapturer API).
3. **Application/task detection:** the window title, process information and captured content are used to identify the supported application and current task.

4. **Information extraction:** relevant content is extracted from the capture (for example through OCR or application-specific parsing; method to be finalized).
5. **Validation:** extracted information is checked using application-specific rules, such as syntax checks for code or formula/format checks for spreadsheets (mechanism to be researched and finalized per application).
6. **AI analysis:** only validated information is sent to an AI model through an API, which returns an explanation and suggestion.
7. **Output:** the result is shown via the avatar, a desktop notification or the chat. If confidence is low, the assistant stays silent.

4.2 Main flow

User → Login → Select application/window → Start monitoring → Application/task detection → Validation → AI analysis → Suggestion/explanation → Avatar / notification / chat

(A block diagram of this flow will be included as Figure 1 in the Appendices.)

4.3 Techniques and libraries (tentative)

- Electron.js desktopCapturer for window capture.
- OCR library such as Tesseract.js, or a vision-capable AI model, for text extraction (to be decided).
- Language-specific tooling such as linters and parsers for validating VS Code content, and document/spreadsheet structure checks for Microsoft Office content (to be decided).
- SQLite for persistent storage; node-cron or an equivalent scheduler for reminders.
- Password hashing (for example bcrypt) for credential security; API keys stored only on the backend.

5. Project Scope

5.1 In scope

The project supports a limited number of selected applications rather than every application on a computer. The final list will be chosen based on technical feasibility, available validation methods and evaluation requirements.

Applications (to be finalized): Visual Studio Code; Microsoft Word, Excel and PowerPoint (final selection to be narrowed); additional application, if feasible.

Supported tasks (to be finalized): three or more specific tasks per selected application, chosen once the validation methods are confirmed.

5.2 Modules

Module	Description
Account management	Registration, login, logout, password recovery, user-specific settings.
Screen monitoring	Window selection, periodic capture, start/stop control.
Detection and extraction	Application and task detection, content extraction.
Validation	Verification of extracted information before AI use.
AI assistant	Mistake explanation, suggestions, chat, confidence-based silence.
Avatar system	Desktop avatar with states and animations, selectable avatars.
Memory	Per-user storage of preferences and information the user asks to remember.
Reminders	One-time, daily and recurring reminders with desktop notifications.
Evaluation	Test scenarios and metric computation.

5.3 Roles

- **User:** the only primary role. Uses all assistant features after logging in.
- **External AI service:** a secondary system actor that receives validated information and returns responses.

5.4 Out of scope

- Support for every application on the computer.
- Autonomous control of the user's mouse or keyboard, or automatic modification of the user's files.
- Monitoring of any window other than the one selected by the user.
- Mobile or web versions.

- Voice interaction, calendar integration and additional productivity tools (possible future work only if the core scope is complete).
- Training a new AI model from scratch.

6. Project Features

6.1 Features of the User role

Account and settings

- Create an account, log in, log out and recover a forgotten password.
- Maintain separate memory, reminders and settings per account.
- Adjust assistant tone and enable or disable playful responses.

Monitoring and task assistance

- Select an application or window to monitor, and start or stop monitoring.
- Automatic detection of the supported application being used.
- Extraction of relevant task information from the selected window.
- Validation of extracted information before it is sent to the AI model.
- Detection of mistakes or issues, with explanations and suggestions.
- The assistant remains silent when it is not sufficiently confident.

Chat and avatar

- Chat with the assistant and ask questions about the current task.
- Interactive desktop avatar shown when the assistant is activated, with states such as sitting, standing and walking.
- Select or add different avatars.
- Messages and notifications displayed through the avatar.

Memory

- Remember information the user explicitly asks it to remember, preferences and useful task information.
- View and delete stored memory.

Reminders

- Create one-time, daily and recurring reminders, for example "Remind me every day at 8 PM to study."
- Receive a desktop notification at the scheduled time.
- View, edit and delete reminders.

Playful mode (optional)

- Occasional jokes about clear mistakes, never when the system is uncertain, with the ability to disable it.

6.2 Use case model

ID	Use case	Actor
UC-01	Register account	User
UC-02	Log in / log out	User
UC-03	Recover password	User
UC-04	Select window to monitor	User
UC-05	Start / stop monitoring	User
UC-06	Receive suggestion or mistake alert	User
UC-07	Chat with assistant	User, AI service
UC-08	Select / add avatar	User
UC-09	Manage memory	User
UC-10	Create / manage reminders	User
UC-11	Receive reminder notification	User
UC-12	Adjust tone and playful mode	User

(A UML use case diagram of these use cases will be included as Figure 2 in the Appendices.)

7. Solution Application Areas

Target domain: personal productivity, education and software development.

- Students and learners:** immediate feedback on coding exercises, documents and spreadsheets, and study reminders.
- Software development:** early detection of mistakes in code opened in VS Code, with explanations that help learning.
- Office work:** guidance on common errors in Word, Excel and PowerPoint tasks.
- General productivity:** a single assistant combining task help, memory and reminders.

Benefits: fewer unnoticed mistakes, less time lost copying content into separate chat tools, more reliable suggestions through validation, user control over what is monitored (privacy), and a measurable evaluation of assistant reliability that can inform future research.

8. Tools / Technology

8.1 Software

Area	Technology
Desktop application	Electron.js
User interface	React.js, Tailwind CSS, JavaScript
Backend	Node.js
Database	SQLite
AI	External AI API (provider to be finalized)
Screen capture / OCR / validation	To be finalized after selecting applications and validation methods
Development tools	VS Code, Git/GitHub, Postman
Evaluation	Scripts for computing precision, recall, F1-score and response time

8.2 Hardware

- Development computers (Windows) with sufficient memory for running Electron and the target applications.
- Stable internet connection for AI API access.

9. References

(Preliminary list; to be extended and verified during the literature review. Review areas: AI desktop assistants, intelligent personal assistants, GUI agents, application/task recognition, error detection, application-specific validation, LLM-based task assistance, HCI, and evaluation metrics.)

[1] S. Yao et al., "ReAct: Synergizing Reasoning and Acting in Language Models," arXiv:2210.03629, 2022.

- [2] T. Xie et al., "OSWorld: Benchmarking Multimodal Agents for Open-Ended Tasks in Real Computer Environments," arXiv:2404.07972, 2024.
- [3] X. Deng et al., "Mind2Web: Towards a Generalist Agent for the Web," NeurIPS, 2023.
- [4] B. Zheng et al., "GPT-4V(ision) is a Generalist Web Agent, if Grounded (SeeAct)," ICML, 2024.
- [5] W. Hong et al., "CogAgent: A Visual Language Model for GUI Agents," CVPR, 2024.
- [6] M. Sokolova and G. Lapalme, "A systematic analysis of performance measures for classification tasks," Information Processing & Management, vol. 45, no. 4, pp. 427-437, 2009.
- [7] Electron Documentation, desktopCapturer API. <https://www.electronjs.org/docs>

10. Appendices

Figure 1: System block diagram (Screen monitoring → Detection → Validation → AI model → Output). *To be inserted.*

Figure 2: Use case diagram. *To be inserted.*

Appendix A: Pending decisions

- Final supported applications and tasks
- Screen capture and application detection methods
- Validation mechanism
- AI API provider and API key security method
- Database structure
- Evaluation dataset, test cases and benchmarking methodology
- Literature review and research gap
- Final project title and FYP scope

Appendix B: Evaluation plan. A test set of predefined task scenarios will be created per supported application. For each scenario, the system's detections will be compared with ground truth to compute precision, recall, F1-score, accuracy where applicable, false positive rate, false negative rate and response time. A comparison between raw-screen suggestions and validated-information suggestions will be used to assess the benefit of the validation layer.